

Inspection of telemetry data with the console

Although the console is not a recommended way to inspect telemetry data, it is a simple and quick way to get started. This article shows you how to output telemetry data to the console for inspection with a minimal Kernel setup.

Exporter

Exporters are responsible for sending telemetry data to a destination. Read more about exporters [here](#) . In this example, we use the console exporter to output telemetry data to the console.

Prerequisites

- An Azure OpenAI chat completion deployment.
- The latest [.Net SDK](#) for your operating system.

Setup

Create a new console application

In a terminal, run the following command to create a new console application in C#:

Console

```
dotnet new console -n TelemetryConsoleQuickstart
```

Navigate to the newly created project directory after the command completes.

Install required packages

- Semantic Kernel

Console

```
dotnet add package Microsoft.SemanticKernel
```

- OpenTelemetry Console Exporter

Console

```
dotnet add package OpenTelemetry.Exporter.Console
```

Create a simple application with Semantic Kernel

From the project directory, open the `Program.cs` file with your favorite editor. We are going to create a simple application that uses Semantic Kernel to send a prompt to a chat completion model. Replace the existing content with the following code and fill in the required values for `deploymentName`, `endpoint`, and `apiKey`:

C#

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using Microsoft.SemanticKernel;
using OpenTelemetry;
using OpenTelemetry.Logs;
using OpenTelemetry.Metrics;
using OpenTelemetry.Resources;
using OpenTelemetry.Trace;

namespace TelemetryConsoleQuickstart
{
    class Program
    {
        static async Task Main(string[] args)
        {
            // Telemetry setup code goes here

            IKernelBuilder builder = Kernel.CreateBuilder();
            // builder.Services.AddSingleton(loggerFactory);
            builder.AddAzureOpenAIChatCompletion(
                deploymentName: "your-deployment-name",
                endpoint: "your-azure-openai-endpoint",
                apiKey: "your-azure-openai-api-key"
            );

            Kernel kernel = builder.Build();

            var answer = await kernel.InvokePromptAsync(
                "Why is the sky blue in one sentence?"
            );

            Console.WriteLine(answer);
        }
    }
}
```

Add telemetry

If you run the console app now, you should expect to see a sentence explaining why the sky is blue. To observe the kernel via telemetry, replace the `// Telemetry setup code goes here` comment with the following code:

C#

```
var resourceBuilder = ResourceBuilder
    .CreateDefault()
    .AddService("TelemetryConsoleQuickstart");

// Enable model diagnostics with sensitive data.
AppContext.SetSwitch("Microsoft.SemanticKernel.Experimental.GenAI.EnableOTelDiagnosticsSensitive", true);

using var traceProvider = Sdk.CreateTracerProviderBuilder()
    .SetResourceBuilder(resourceBuilder)
    .AddSource("Microsoft.SemanticKernel*")
    .AddConsoleExporter()
    .Build();

using var meterProvider = Sdk.CreateMeterProviderBuilder()
    .SetResourceBuilder(resourceBuilder)
    .AddMeter("Microsoft.SemanticKernel*")
    .AddConsoleExporter()
    .Build();

using var loggerFactory = LoggerFactory.Create(builder =>
{
    // Add OpenTelemetry as a logging provider
    builder.AddOpenTelemetry(options =>
    {
        options.SetResourceBuilder(resourceBuilder);
        options.AddConsoleExporter();
        // Format log messages. This is default to false.
        options.IncludeFormattedMessage = true;
        options.IncludeScopes = true;
    });
    builder.SetMinimumLevel(LogLevel.Information);
});
```

Finally Uncomment the line `// builder.Services.AddSingleton(loggerFactory);` to add the logger factory to the builder.

In the above code snippet, we first create a resource builder for building resource instances. A resource represents the entity that produces telemetry data. You can read more about resources [here](#) . The resource builder to the providers is optional. If not provided, the default resource with default attributes is used.

Next, we turn on diagnostics with sensitive data. This is an experimental feature that allows you to enable diagnostics for the AI services in the Semantic Kernel. With this turned on, you will see additional telemetry data such as the prompts sent to and the responses received from the AI models, which are considered sensitive data. If you don't want to include sensitive data in your telemetry, you can use another switch

`Microsoft.SemanticKernel.Experimental.GenAI.EnableOTelDiagnostics` to enable diagnostics with non-sensitive data, such as the model name, the operation name, and token usage, etc.

Then, we create a tracer provider builder and a meter provider builder. A provider is responsible for processing telemetry data and piping it to exporters. We subscribe to the `Microsoft.SemanticKernel*` source to receive telemetry data from the Semantic Kernel namespaces. We add a console exporter to both the tracer provider and the meter provider. The console exporter sends telemetry data to the console.

Finally, we create a logger factory and add `OpenTelemetry` as a logging provider that sends log data to the console. We set the minimum log level to `Information` and include formatted messages and scopes in the log output. The logger factory is then added to the builder.

Important

A provider should be a singleton and should be alive for the entire application lifetime. The provider should be disposed of when the application is shutting down.

Run

Run the console application with the following command:

Console

```
dotnet run
```

Inspect telemetry data

Log records

You should see multiple log records in the console output. They look similar to the following:

Console

```
LogRecord.Timestamp:          2024-09-12T21:48:35.2295938Z
LogRecord.TraceId:           159d3f07664838f6abdad7af6a892cfa
LogRecord.SpanId:            ac79a006da8a6215
LogRecord.TraceFlags:        Recorded
LogRecord.CategoryName:      Microsoft.SemanticKernel.KernelFunction
LogRecord.Severity:          Info
LogRecord.SeverityText:      Information
LogRecord.FormattedMessage:  Function
InvokePromptAsync_290eb9bece084b00aea46b569174feae invoking.
LogRecord.Body:              Function {FunctionName} invoking.
LogRecord.Attributes (Key:Value):
    FunctionName: InvokePromptAsync_290eb9bece084b00aea46b569174feae
    OriginalFormat (a.k.a Body): Function {FunctionName} invoking.
```

```
Resource associated with LogRecord:
service.name: TelemetryConsoleQuickstart
service.instance.id: a637dfc9-0e83-4435-9534-fb89902e64f8
telemetry.sdk.name: opentelemetry
telemetry.sdk.language: dotnet
telemetry.sdk.version: 1.9.0
```

There are two parts to each log record:

- The log record itself: contains the timestamp and namespace at which the log record was generated, the severity and body of the log record, and any attributes associated with the log record.
- The resource associated with the log record: contains information about the service, instance, and SDK used to generate the log record.

Activities

📌 Note

Activities in .Net are similar to spans in OpenTelemetry. They are used to represent a unit of work in the application.

You should see multiple activities in the console output. They look similar to the following:

Console

```
Activity.TraceId:          159d3f07664838f6abdad7af6a892cfa
Activity.SpanId:           8c7c79bc1036eab3
Activity.TraceFlags:       Recorded
Activity.ParentSpanId:     ac79a006da8a6215
Activity.ActivitySourceName: Microsoft.SemanticKernel.Diagnostics
Activity.DisplayName:      chat.completions gpt-4o
```

```
Activity.Kind: Client
Activity.StartTime: 2024-09-12T21:48:35.5717463Z
Activity.Duration: 00:00:02.3992014
Activity.Tags:
  gen_ai.operation.name: chat.completions
  gen_ai.system: openai
  gen_ai.request.model: gpt-4o
  gen_ai.response.prompt_tokens: 16
  gen_ai.response.completion_tokens: 29
  gen_ai.response.finish_reason: Stop
  gen_ai.response.id: chatcmpl-A6lxxz14rKuQpQibmiCpzmye6z9rxC
Activity.Events:
  gen_ai.content.prompt [9/12/2024 9:48:35 PM +00:00]
    gen_ai.prompt: [{"role": "user", "content": "Why is the sky blue in one sentence?"}]
  gen_ai.content.completion [9/12/2024 9:48:37 PM +00:00]
    gen_ai.completion: [{"role": "Assistant", "content": "The sky appears blue because shorter blue wavelengths of sunlight are scattered in all directions by the gases and particles in the Earth's atmosphere more than other colors."}]
Resource associated with Activity:
  service.name: TelemetryConsoleQuickstart
  service.instance.id: a637dfc9-0e83-4435-9534-fb89902e64f8
  telemetry.sdk.name: opentelemetry
  telemetry.sdk.language: dotnet
  telemetry.sdk.version: 1.9.0
```

There are two parts to each activity:

- The activity itself: contains the span ID and parent span ID that APM tools use to build the traces, the duration of the activity, and any tags and events associated with the activity.
- The resource associated with the activity: contains information about the service, instance, and SDK used to generate the activity.

Important

The attributes to pay extra attention to are the ones that start with `gen_ai`. These are the attributes specified in the [GenAI Semantic Conventions](#).

Metrics

You should see multiple metric records in the console output. They look similar to the following:

Console

```
Metric Name: semantic_kernel.connectors.openai.tokens.prompt, Number of prompt tokens used, Unit: {token}, Meter: Microsoft.SemanticKernel.Connectors.OpenAI
```

```
(2024-09-12T21:48:37.9531072Z, 2024-09-12T21:48:38.0966737Z] LongSum  
Value: 16
```

Here you can see the name, the description, the unit, the time range, the type, the value of the metric, and the meter that the metric belongs to.

ⓘ Note

The above metric is a Counter metric. For a full list of metric types, see [here](#). Depending on the type of metric, the output may vary.

Next steps

Now that you have successfully output telemetry data to the console, you can learn more about how to use APM tools to visualize and analyze telemetry data.

Application Insights

Last updated on 09/25/2024